RELEASE CANDIDATE

Context-Fabric 0.6.0rc1

The Text-Fabric API on a new Rust core: same answers, a fraction of the memory, several times faster.

Context-Fabric reads Text-Fabric corpora and keeps the Text-Fabric Python API (`F`, `E`, `L`, `T`, `S`, `N`, `C`). In 0.6 the engine is rewritten in Rust and serves every query from a compiled, memory-mapped cache, so loading is nearly instant and memory stays small.

33 MB
Memory after loading
Text-Fabric: 6,170 MB

0.003 s
Load time
Text-Fabric: 4.0 s

15×
Faster search
typical speedup over 100 benchmark queries

34/34
Identical query results
full result sets match Text-Fabric

WHAT YOU GET

- **Drop-in API.** Existing Text-Fabric scripts run unchanged: the same calls, arguments and return values for the core functions.
- **Same answers.** Checked against Text-Fabric 13.0.19 on three corpora: about 505,000 value checks, the 34-query search suite, and recorded reference outputs.
- **Small, shared memory.** The corpus stays memory-mapped, so worker processes share one copy. Four BHSA workers use 642 MB, where Text-Fabric uses 25 GB.
- **Safe search.** Results stream lazily, runaway searches stop at Text-Fabric's result cap, Ctrl-C interrupts a running search, and a bad template raises a clear `SearchTemplateError`.
- **Light install.** No required dependencies. Optional extras: `[numpy]` for array output, `[download]` for fetching corpora.

PER-CALL SPEED (BHSA)

CALL	TF 13.0	CF 0.5	CF 0.6
`F.sp.v`	0.056 µs	0.183 µs	**0.029 µs**
`L.d`	0.57 µs	3.27 µs	**0.14 µs**
`L.n`	0.24 µs	1.58 µs	**0.11 µs**
`E.mother.t`	0.043 µs	0.389 µs	**0.030 µs**
`T.sectionFromNode`	5.3 µs	13.8 µs	**0.45 µs**
`T.text` (verse)	8.8 µs	31.3 µs	**1.6 µs**
34-query suite	12.9 s	27.6 s	**2.2 s**

TF: Text-Fabric. CF 0.5: the previous pure-Python Context-Fabric. Median of repeated calls on a warm cache. CF 0.6 is also faster than Text-Fabric on every one of the 34 parity queries.

```
pip install --pre "context-fabric==0.6.0rc1"
```

Python 3.13+ · wheels for Linux, macOS and Windows

Good to know. The first load compiles the corpus into a cache (about 29 s and 614 MB on disk for BHSA); every later load reuses it. Caches from 0.5 are not read; they are rebuilt automatically. `L.u` (0.13 vs 0.11 µs) and a full `N.walk()` are slightly slower than Text-Fabric. A short list of deliberate differences, such as tuples instead of arrays for some row data, is in the changelog. This is a release candidate: please report anything that differs from Text-Fabric.

Measured on BHSA (Hebrew Bible, 1.4M nodes) against Text-Fabric 13.0.19 and Context-Fabric 0.5.8, Apple M5 Max, Python 3.13. Method and full results: libs/benchmarks/baselines/RC1_BASELINES.md.

github.com/Context-Fabric/context-fabric